

OBSERVATION TASK 2

Course :	Full Stack Web Development
Course Code:	23CM4121
Observation task no.:	2
Student Name:	R. Srivalli
Roll No.:	A24126552108

1. TITLE

Student Profile Manager using JavaScript Class and DOM Manipulation

2. AIM

To develop a **Student Profile Manager** using JavaScript classes and DOM manipulation that accepts student details such as **Name, Roll Number, Department, and CGPA**, creates a Student object, and dynamically displays the student profile on the webpage when the button is clicked.

3. OBJECTIVE

- To understand the concept of **JavaScript classes and objects**.
 - To create a Student class with properties such as **Name, Roll Number, Department, and CGPA**.
 - To create a **Student object** using values entered by the user.
 - To understand and implement **DOM selection and manipulation**.
 - To dynamically create HTML elements using JavaScript.
 - To display student details dynamically on the webpage.
 - To handle user interaction using **event handling**.
 - To apply **CSS styling** to the dynamically generated student profile.
-

4. THEORY

JavaScript Classes and Objects:

JavaScript classes provide a structured way to create objects with related properties and methods. In this application, a Student class is created with properties such as Name, Roll Number, Department, and CGPA. A Student object is then created using the values entered by the user.

DOM Manipulation:

The Document Object Model (DOM) represents the webpage as a collection of objects that can be accessed and modified using JavaScript. DOM methods such as `getElementById()`, `createElement()`, and `appendChild()` are used to select existing elements and dynamically create new elements.

Event Handling:

An event listener is used to detect the form submission. When the user clicks the Generate Profile button, JavaScript reads the entered student details, creates a Student object, and dynamically generates the profile on the webpage without reloading the page.

Dynamic Content:

The student profile is created using JavaScript elements and displayed using `textContent`. The CSS file provides styling for the generated profile, including layout, colors, spacing, and responsive design. Thus, the application demonstrates the practical use of JavaScript classes, objects, DOM manipulation, and event handling.

5. ALGORITHM / PROCEDURE

1. Create an HTML webpage containing input fields for **Name, Roll Number, Department, and CGPA**.
 2. Create a **Student class** in JavaScript with properties for the student details.
 3. Define methods in the class to generate the student's initials and profile description.
 4. Select the form and profile display area using **DOM selection methods**.
 5. Add a **submit event listener** to the form.
 6. Read the values entered by the user and remove unnecessary spaces.
 7. Create a Student object using the entered values.
 8. Clear the existing profile area.
 9. Dynamically create HTML elements using `createElement()`.
 10. Add student information to the created elements using `textContent`.
 11. Apply CSS classes using `classList.add()` and arrange the elements using `appendChild()`.
 12. Display the generated student profile on the webpage.
 13. Format the CGPA to two decimal places using `toFixed(2)`.
 14. Test the application with valid and invalid inputs to verify its functionality.
-

6. PROGRAM

A : Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Student Spotlight</title>

<link rel="stylesheet" href="styleobtask2.css">
</head>

<body>

  <main class="page-wrapper">

    <!-- Left side: Introduction -->
    <section class="intro-panel">

      <div class="brand">
        <span class="brand-mark">SP</span>
        <span>Student Spotlight</span>
      </div>

      <div class="intro-content">
        <p class="tag">JAVASCRIPT • DOM</p>

        <h1>
          Create your
          <span>student profile.</span>
        </h1>

        <p class="intro-text">
          Enter your academic details and generate a
          personalized student profile using JavaScript
          classes and DOM manipulation.
        </p>
      </div>

      <div class="tech-note">
        <span>JS CLASS</span>
        <span>DOM</span>
        <span>EVENTS</span>
      </div>

    </section>

    <!-- Right side: Form and generated profile -->
    <section class="workspace">

      <div class="form-box">

        <div class="section-heading">
          <div>
            <p class="small-title">PROFILE BUILDER</p>
            <h2>Student Details</h2>
          </div>


```

```
<span class="step-number">01</span>
</div>

<form id="studentForm">

  <div class="input-group">
    <label for="studentName">Full Name</label>
    <input
      type="text"
      id="studentName"
      placeholder="e.g. Ravi Kumar"
      required
    >
  </div>

  <div class="input-group">
    <label for="rollNumber">Roll Number</label>
    <input
      type="text"
      id="rollNumber"
      placeholder="e.g. A23126552001"
      required
    >
  </div>

  <div class="two-columns">

    <div class="input-group">
      <label for="department">Department</label>
      <input
        type="text"
        id="department"
        placeholder="e.g. CSE (AI & ML)"
        required
      >
    </div>

    <div class="input-group">
      <label for="cgpa">CGPA</label>
      <input
        type="number"
        id="cgpa"
        min="0"
        max="10"
        step="0.01"
        placeholder="8.42"
        required
      >
    </div>

  </div>
```

```

        <button type="submit" id="generateButton">
            Generate Profile
        <span>→</span>
        </button>

    </form>

</div>

<!-- Dynamic profile will appear here -->
<div id="profileArea">

    <div class="empty-state">
        <div class="empty-icon">+</div>

        <h3>Your profile will appear here</h3>

        <p>
            Fill in the details and click
            <strong>Generate Profile</strong>.
        </p>
    </div>

</div>

</section>

</main>

<footer>
    <p>Observation Task 2 • JavaScript Class & DOM Manipulation</p>
</footer>

<script src="scriptobstask2.js"></script>

</body>
</html>

```

B : style.css

```

/* -----
Student Spotlight
Custom CSS Design
----- */

/* Basic reset */
* {
    margin: 0;
    padding: 0;

```

```

    box-sizing: border-box;
}

/* Page */
body {
    min-height: 100vh;
    font-family: Arial, Helvetica, sans-serif;
    background: #e9e7df;
    color: #20201e;
}

/* Main layout */
.page-wrapper {
    width: 92%;
    max-width: 1150px;
    min-height: 650px;
    margin: 45px auto 20px;

    display: grid;
    grid-template-columns: 0.9fr 1.1fr;

    background: #ffffff;
    border-radius: 18px;
    overflow: hidden;

    box-shadow: 0 18px 50px rgba(0, 0, 0, 0.12);
}

/* -----
Introduction Panel
----- */

.intro-panel {
    background: #20201e;
    color: white;

    padding: 42px;

    display: flex;
    flex-direction: column;
    justify-content: space-between;
}

.brand {
    display: flex;
    align-items: center;
    gap: 12px;

    font-size: 17px;
    font-weight: bold;
}

```

```
.brand-mark {
  width: 38px;
  height: 38px;

  display: flex;
  align-items: center;
  justify-content: center;

  border: 1px solid #ffffff;
  border-radius: 50%;

  font-size: 12px;
  letter-spacing: 1px;
}

.intro-content {
  max-width: 420px;
}

.tag {
  margin-bottom: 18px;

  font-size: 12px;
  letter-spacing: 3px;

  color: #c7a86b;
}

.intro-content h1 {
  font-size: clamp(40px, 5vw, 65px);
  line-height: 1.02;
  font-weight: 700;
}

.intro-content h1 span {
  display: block;
  color: #c7a86b;
}

.intro-text {
  margin-top: 25px;

  max-width: 390px;

  color: #c9c9c5;

  line-height: 1.7;
  font-size: 15px;
}
```

```
.tech-note {
  display: flex;
  gap: 10px;
}

.tech-note span {
  padding: 7px 10px;

  border: 1px solid #555550;
  border-radius: 20px;

  color: #c9c9c5;

  font-size: 10px;
  letter-spacing: 1px;
}

/* -----
Workspace
----- */

.workspace {
  padding: 42px;

  background: #faf9f5;
}

.form-box {
  max-width: 650px;
  margin: auto;
}

.section-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;

  margin-bottom: 28px;
}

.small-title {
  margin-bottom: 5px;

  color: #9a7a3f;

  font-size: 11px;
  font-weight: bold;
  letter-spacing: 2px;
}

.section-heading h2 {
```

```
    font-size: 28px;
  }

  .step-number {
    color: #aaa69c;

    font-size: 13px;
    letter-spacing: 2px;
  }

  /* -----
  Form
  ----- */

  .input-group {
    margin-bottom: 18px;
  }

  .input-group label {
    display: block;

    margin-bottom: 7px;

    font-size: 13px;
    font-weight: bold;
  }

  .input-group input {
    width: 100%;

    padding: 13px 14px;

    border: 1px solid #d4d1c8;
    border-radius: 8px;

    background: white;

    font-size: 14px;

    transition: 0.25s ease;
  }

  .input-group input:focus {
    outline: none;

    border-color: #9a7a3f;

    box-shadow: 0 0 3px rgba(154, 122, 63, 0.10);
  }

  .two-columns {
```

```

display: grid;
grid-template-columns: 1fr 1fr;
gap: 16px;
}

/* Button */
#generateButton {
  width: 100%;

  margin-top: 5px;
  padding: 14px 18px;

  border: none;
  border-radius: 8px;

  background: #20201e;
  color: white;

  font-size: 14px;
  font-weight: bold;

  cursor: pointer;

  display: flex;
  align-items: center;
  justify-content: space-between;

  transition: 0.25s ease;
}

#generateButton:hover {
  background: #9a7a3f;

  transform: translateY(-2px);
}

#generateButton span {
  font-size: 20px;
}

/* -----
Empty State
----- */

#profileArea {
  margin-top: 35px;
}

.empty-state {
  padding: 30px;

```

```
text-align: center;

border: 1px dashed #c9c5bb;
border-radius: 12px;

color: #77746d;
}

.empty-icon {
width: 42px;
height: 42px;

margin: 0 auto 12px;

display: flex;
align-items: center;
justify-content: center;

border-radius: 50%;

background: #ebe7dd;

font-size: 24px;
}

.empty-state h3 {
margin-bottom: 7px;

color: #44423d;
font-size: 16px;
}

.empty-state p {
font-size: 13px;
}

/* -----
Dynamic Student Profile
----- */

.student-profile {
position: relative;

padding: 28px;

background: #20201e;
color: white;

border-radius: 14px;

overflow: hidden;
```

```
    animation: profileAppear 0.4s ease;
  }

.student-profile::after {
  content: "";

  position: absolute;

  width: 170px;
  height: 170px;

  right: -70px;
  top: -70px;

  border-radius: 50%;

  background: rgba(199, 168, 107, 0.15);
}

.profile-top {
  position: relative;
  z-index: 1;

  display: flex;
  align-items: center;
  gap: 18px;

  padding-bottom: 22px;

  border-bottom: 1px solid #44443f;
}

.initials {
  width: 62px;
  height: 62px;

  flex-shrink: 0;

  display: flex;
  align-items: center;
  justify-content: center;

  border-radius: 50%;

  background: #c7a86b;
  color: #20201e;

  font-size: 20px;
  font-weight: bold;
}
```

```
.profile-heading h3 {
  margin-bottom: 5px;

  font-size: 22px;
}

.profile-heading p {
  color: #bdbdb8;

  font-size: 12px;
}

.profile-status {
  margin-left: auto;

  padding: 6px 9px;

  border: 1px solid #555550;
  border-radius: 20px;

  color: #c7a86b;

  font-size: 9px;
  letter-spacing: 1px;
}

/* Profile details */
.profile-details {
  position: relative;
  z-index: 1;

  display: grid;
  grid-template-columns: 1fr 1fr;

  gap: 18px;

  padding-top: 23px;
}

.detail {
  padding: 14px;

  background: #292925;

  border-radius: 8px;
}

.detail-label {
  display: block;
```

```

margin-bottom: 6px;

color: #969690;

font-size: 10px;
letter-spacing: 1px;
text-transform: uppercase;
}

.detail-value {
  color: white;

  font-size: 14px;
  font-weight: bold;
}

.cgpa-value {
  color: #c7a86b;

  font-size: 19px;
}

/* Animation */
@keyframes profileAppear {

  from {
    opacity: 0;
    transform: translateY(10px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* -----
Footer
----- */

footer {
  text-align: center;

  padding: 10px 20px 25px;

  color: #77746d;

  font-size: 12px;
}

/* -----

```

Responsive Design

```
----- */

@media (max-width: 800px) {

  .page-wrapper {
    grid-template-columns: 1fr;

    margin-top: 20px;
  }

  .intro-panel {
    min-height: 430px;
  }

  .workspace {
    padding: 30px 22px;
  }
}

@media (max-width: 550px) {

  .page-wrapper {
    width: 95%;
  }

  .intro-panel {
    padding: 30px 24px;
  }

  .workspace {
    padding: 25px 18px;
  }

  .two-columns {
    grid-template-columns: 1fr;
    gap: 0;
  }

  .profile-details {
    grid-template-columns: 1fr;
  }

  .profile-status {
    display: none;
  }

  .intro-content h1 {
    font-size: 42px;
  }
}
```

C : script.js

```
/*
  Observation Task 2
  Student Profile Manager
  Using JavaScript Class and DOM
*/

// -----
// Student Class
// -----

class Student {

  constructor(name, rollNumber, department, cgpa) {

    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = Number(cgpa);
  }

  // Returns the student's initials
  getInitials() {

    const words = this.name.trim().split(/\s+/);

    if (words.length === 1) {
      return words[0].substring(0, 2).toUpperCase();
    }

    return (
      words[0].charAt(0) +
      words[words.length - 1].charAt(0)
    ).toUpperCase();
  }

  // Creates a short description for the profile
  getDescription() {

    return `${this.department} • Academic Profile`;
  }
}

// -----
// DOM Selection
// -----

const studentForm = document.getElementById("studentForm");
const profileArea = document.getElementById("profileArea");

// -----
```

```

// Generate Student Profile
// -----

studentForm.addEventListener("submit", function (event) {

    // Prevent the form from refreshing the page
    event.preventDefault();

    // Read user input
    const name = document.getElementById("studentName").value.trim();
    const rollNumber = document.getElementById("rollNumber").value.trim();
    const department = document.getElementById("department").value.trim();
    const cgpa = document.getElementById("cgpa").value;

    // Create Student object
    const student = new Student(
        name,
        rollNumber,
        department,
        cgpa
    );

    // Clear the previous content
    profileArea.innerHTML = "";

    // Create main profile container
    const profile = document.createElement("div");
    profile.classList.add("student-profile");

    // -----
    // Profile Header
    // -----

    const profileTop = document.createElement("div");
    profileTop.classList.add("profile-top");

    // Student initials
    const initials = document.createElement("div");
    initials.classList.add("initials");
    initials.textContent = student.getInitials();

    // Heading area
    const profileHeading = document.createElement("div");
    profileHeading.classList.add("profile-heading");

    const heading = document.createElement("h3");
    heading.textContent = student.name;

    const description = document.createElement("p");
    description.textContent = student.getDescription();

```

```

profileHeading.appendChild(heading);
profileHeading.appendChild(description);

// Status badge
const status = document.createElement("span");
status.classList.add("profile-status");
status.textContent = "GENERATED";

// Add header elements
profileTop.appendChild(initials);
profileTop.appendChild(profileHeading);
profileTop.appendChild(status);

// -----
// Profile Details
// -----

const details = document.createElement("div");
details.classList.add("profile-details");

// Helper function for creating detail boxes
function createDetail(label, value, specialClass = "") {

    const detail = document.createElement("div");
    detail.classList.add("detail");

    const labelElement = document.createElement("span");
    labelElement.classList.add("detail-label");
    labelElement.textContent = label;

    const valueElement = document.createElement("span");
    valueElement.classList.add("detail-value");

    if (specialClass !== "") {
        valueElement.classList.add(specialClass);
    }

    valueElement.textContent = value;

    detail.appendChild(labelElement);
    detail.appendChild(valueElement);

    return detail;
}

// Add student information
details.appendChild(
    createDetail("Roll Number", student.rollNumber)
);

details.appendChild(

```

```
        createDetail("Department", student.department)
    );

    details.appendChild(
        createDetail("Academic Status", "Active Student")
    );

    details.appendChild(
        createDetail(
            "CGPA",
            student.cgpa.toFixed(2),
            "cgpa-value"
        )
    );

    // -----
    // Add Everything to the Profile
    // -----

    profile.appendChild(profileTop);
    profile.appendChild(details);

    profileArea.appendChild(profile);

});
```

7. CODE EXPLANATION

S.No.	Code / Component	Explanation
1	class Student	Defines a JavaScript class for storing student information.
2	constructor(name, rollNumber, department, cgpa)	Initializes the Student object with the values provided by the user.
3	this.name	Stores the student's name.
4	this.rollNumber	Stores the student's roll number.
5	this.department	Stores the student's department.
6	this.cgpa = Number(cgpa)	Converts the entered CGPA value into a number.
7	getInitials()	Generates the student's initials from the entered name.

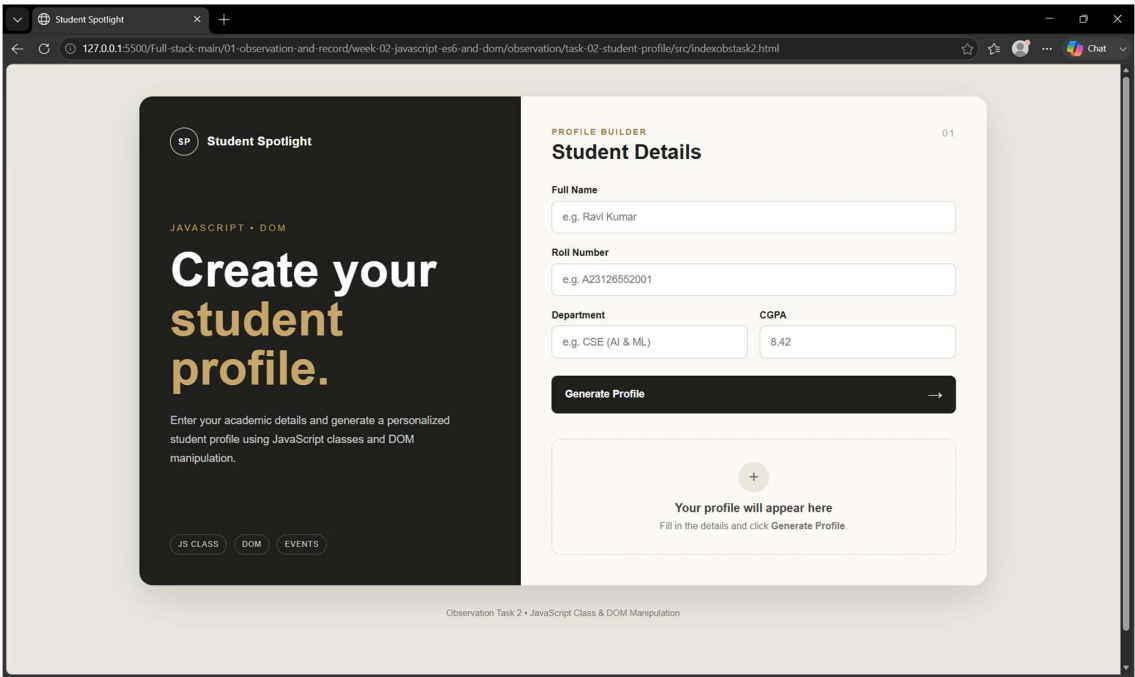
8	getDescription()	Returns a description containing the department and academic profile.
9	document.getElementById()	Selects the required HTML elements from the DOM.
10	addEventListener("submit", ...)	Handles the form submission event when the Generate Profile button is clicked.
11	event.preventDefault()	Prevents the webpage from refreshing after form submission.
12	.value.trim()	Reads user input and removes unnecessary spaces.
13	new Student(...)	Creates a Student object using the entered values.
14	profileArea.innerHTML = ""	Clears any previously generated student profile.
15	document.createElement()	Dynamically creates HTML elements using JavaScript.
16	classList.add()	Adds CSS classes to the dynamically created elements.
17	textContent	Inserts student information into the created HTML elements.
18	appendChild()	Adds the dynamically created elements to their parent elements.
19	createDetail()	Creates reusable detail elements for Roll Number, Department, Status, and CGPA.
20	toFixed(2)	Displays the CGPA with exactly two decimal places.
21	.student-profile	CSS class used to style the dynamically generated student profile.
22	@keyframes profileAppear	Creates an animation when the student profile appears.
23	@media	Makes the webpage responsive on tablets and mobile devices.

8. TEST CASES AND EXECUTION DATA

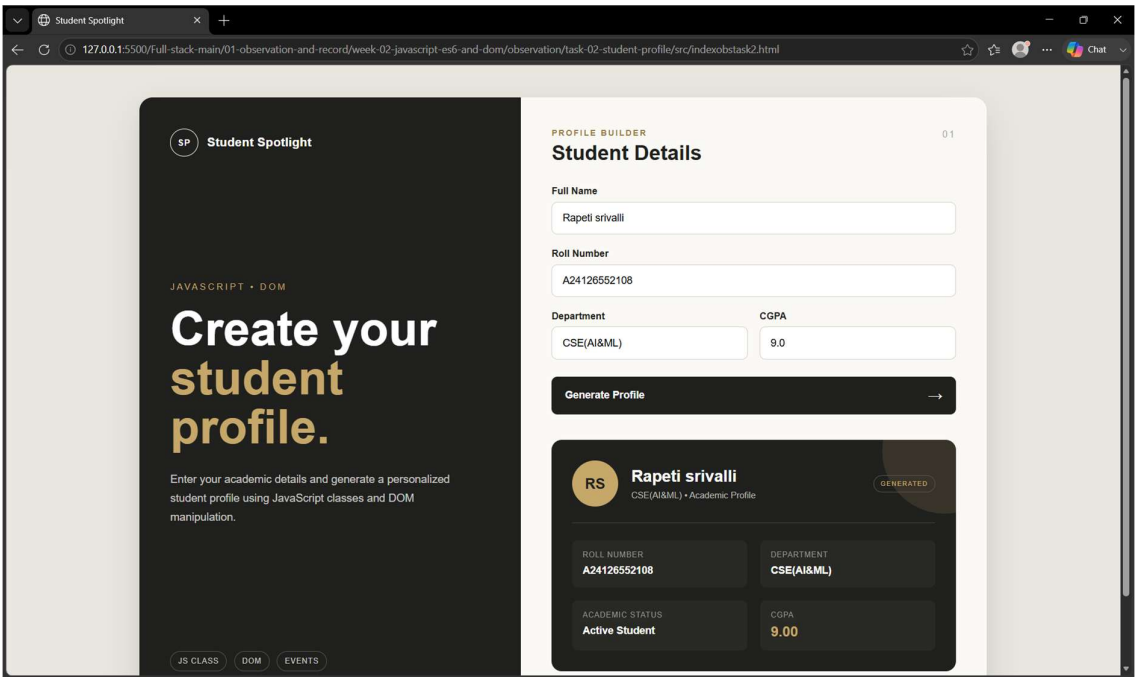
Test Case	Input	Action	Expected Result	Status
TC01	Valid webpage	Open the HTML file in a browser	Student Profile Manager loads successfully	Pass
TC02	Name: Ravi Kumar, Roll No: A23126552001, Department: CSE (AI & ML), CGPA: 8.42	Click Generate Profile	Student profile is dynamically displayed	Pass
TC03	Name: Ravi Kumar	Click Generate Profile	Initials are generated and displayed in the profile	Pass
TC04	CGPA: 8.42	Generate profile	CGPA is displayed as 8.42	Pass
TC05	Enter a second student's details	Click Generate Profile again	Previous profile is cleared and the new profile is displayed	Pass
TC06	Leave required fields empty	Click Generate Profile	Browser validation prevents form submission	Pass
TC07	CGPA: 11	Click Generate Profile	Browser rejects the value because CGPA must be between 0 and 10	Pass
TC08	Valid Name, Roll No, Department and CGPA	Generate profile	All entered details are displayed correctly in the profile	Pass

9. OUTPUT

Output 1: Initial Webpage



Output 2: Generated Student Profile



10. RESULT

The Student Profile Manager was successfully developed using JavaScript classes and DOM manipulation. The application accepts student details, creates a Student object, and dynamically generates and displays the student profile with appropriate CSS styling.